

```
#> 6      Brazil 1999 population 172006362
#> # ... with 6 more rows
```

To tidy this up, we first analyze the representation in a similar way to `gather()`. This time, however, we only need two parameters:

- The column that contains variable names, the key column. Here, it's `type`.
- The column that contains values forms multiple variables, the value column. Here, it's `count`.

Once we've figured that out, we can use `spread()`, as shown programmatically here, and visually in [Figure 9-3](#):

```
spread(table2, key = type, value = count)
#> # A tibble: 6 × 4
#>   country year cases population
#> *   <chr>   <int> <int>   <int>
#> 1 Afghanistan 1999    745  19987071
#> 2 Afghanistan 2000   2666  20595360
#> 3      Brazil 1999  37737  172006362
#> 4      Brazil 2000  80488  174504898
#> 5      China 1999 212258 1272915272
#> 6      China 2000 213766 1280428583
```

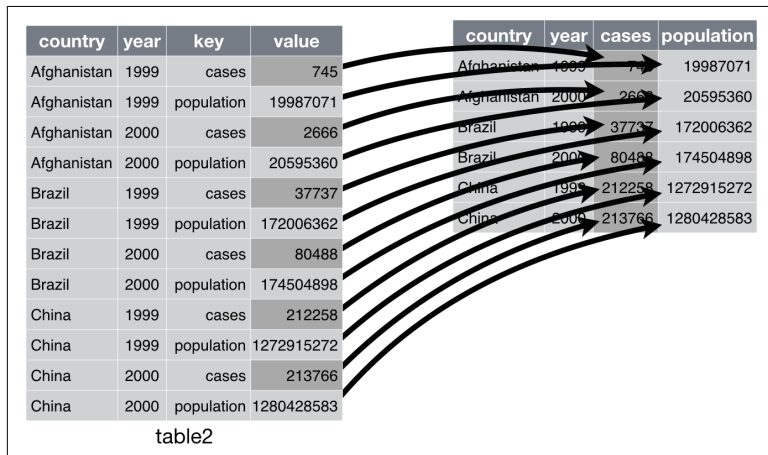


Figure 9-3. Spreading `table2` makes it tidy

As you might have guessed from the common `key` and `value` arguments, `spread()` and `gather()` are complements. `gather()` makes wide tables narrower and longer; `spread()` makes long tables shorter and wider.

Exercises

1. Why are `gather()` and `spread()` not perfectly symmetrical? Carefully consider the following example:

```
stocks <- tibble(
  year   = c(2015, 2015, 2016, 2016),
  half   = c( 1,    2,    1,    2),
  return = c(1.88, 0.59, 0.92, 0.17)
)
stocks %>%
  spread(year, return) %>%
  gather("year", "return", `2015`:`2016`)
```

(Hint: look at the variable types and think about column names.)

Both `spread()` and `gather()` have a `convert` argument. What does it do?

2. Why does this code fail?

```
table4a %>%
  gather(1999, 2000, key = "year", value = "cases")
#> Error in eval(expr, envir, enclos):
#> Position must be between 0 and n
```

3. Why does spreading this tibble fail? How could you add a new column to fix the problem?

```
people <- tribble(
  ~name,          ~key,    ~value,
  #-----/-----/-----
  "Phillip Woods", "age",    45,
  "Phillip Woods", "height", 186,
  "Phillip Woods", "age",    50,
  "Jessica Cordero", "age",    37,
  "Jessica Cordero", "height", 156
)
```

4. Tidy this simple tibble. Do you need to spread or gather it? What are the variables?

```
preg <- tribble(
  ~pregnant, ~male, ~female,
  "yes",     NA,    10,
  "no",      20,    12
)
```

Separating and Pull

So far you've learned how to tidy `table2` and `table4`, but not `table3`. `table3` has a different problem: we have one column (`rate`) that contains two variables (`cases` and `population`). To fix this problem, we'll need the `separate()` function. You'll also learn about the complement of `separate()`: `unite()`, which you use if a single variable is spread across multiple columns.

Separate

`separate()` pulls apart one column into multiple columns, by splitting wherever a separator character appears. Take `table3`:

```
table3
#> # A tibble: 6 × 3
#>   country year      rate
#> *   <chr> <int>    <chr>
#> 1 Afghanistan 1999 745/19987071
#> 2 Afghanistan 2000 2666/20595360
#> 3   Brazil 1999 37737/172006362
#> 4   Brazil 2000 80488/174504898
#> 5    China 1999 212258/1272915272
#> 6    China 2000 213766/1280428583
```

The `rate` column contains both `cases` and `population` variables, and we need to split it into two variables. `separate()` takes the name of the column to separate, and the names of the columns to separate into, as shown in [Figure 9-4](#) and the following code:

```
table3 %>%
  separate(rate, into = c("cases", "population"))
#> # A tibble: 6 × 4
#>   country year cases population
#> *   <chr> <int> <chr>    <chr>
#> 1 Afghanistan 1999 745 19987071
#> 2 Afghanistan 2000 2666 20595360
#> 3   Brazil 1999 37737 172006362
#> 4   Brazil 2000 80488 174504898
#> 5    China 1999 212258 1272915272
#> 6    China 2000 213766 1280428583
```

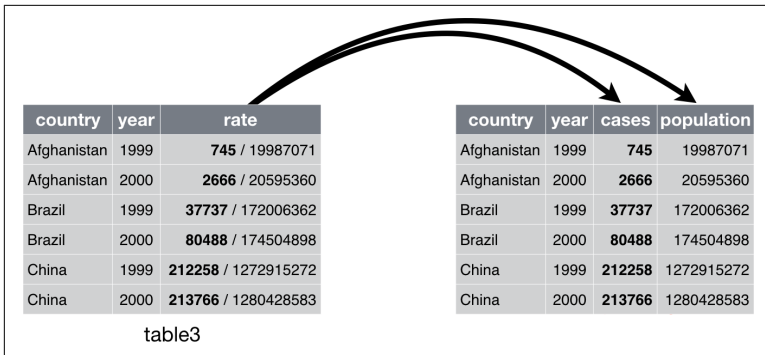


Figure 9-4. Separating table3 makes it tidy

By default, `separate()` will split values wherever it sees a non-alphanumeric character (i.e., a character that isn't a number or letter). For example, in the preceding code, `separate()` split the values of `rate` at the forward slash characters. If you wish to use a specific character to separate a column, you can pass the character to the `sep` argument of `separate()`. For example, we could rewrite the preceding code as:

```
table3 %>%
  separate(rate, into = c("cases", "population"), sep = "/")
```

(Formally, `sep` is a regular expression, which you'll learn more about in [Chapter 11](#).)

Look carefully at the column types: you'll notice that `case` and `popu` `lation` are character columns. This is the default behavior in `separate()`: it leaves the type of the column as is. Here, however, it's not very useful as those really are numbers. We can ask `separate()` to try and convert to better types using `convert = TRUE`:

```
table3 %>%
  separate(
    rate,
    into = c("cases", "population"),
    convert = TRUE
  )
#> # A tibble: 6 × 4
#>   country year cases population
#>   *      <chr> <int> <int>    <int>
#> 1 Afghanistan 1999   745  19987071
#> 2 Afghanistan 2000  2666  20595360
#> 3   Brazil 1999 37737 172006362
#> 4   Brazil 2000 80488 174504898
```

```
#> 5      China 1999 212258 1272915272
#> 6      China 2000 213766 1280428583
```

You can also pass a vector of integers to `sep`. `separate()` will interpret the integers as positions to split at. Positive values start at 1 on the far left of the strings; negative values start at -1 on the far right of the strings. When using integers to separate strings, the length of `sep` should be one less than the number of names in `into`.

You can use this arrangement to separate the last two digits of each year. This makes this data less tidy, but is useful in other cases, as you'll see in a little bit:

```
table3 %>%
  separate(year, into = c("century", "year"), sep = 2)
#> # A tibble: 6 × 4
#>   country century year      rate
#>   *      <chr>   <chr> <chr>   <chr>
#> 1 Afghanistan 19  99    745/19987071
#> 2 Afghanistan 20  00    2666/20595360
#> 3   Brazil    19  99    37737/172006362
#> 4   Brazil    20  00    80488/174504898
#> 5     China   19  99   212258/1272915272
#> 6     China   20  00   213766/1280428583
```

Unite

`unite()` is the inverse of `separate()`: it combines multiple columns into a single column. You'll need it much less frequently than `separate()`, but it's still a useful tool to have in your back pocket.

We can use `unite()` to rejoin the `century` and `year` columns that we created in the last example. That data is saved as `tidyr::table5`. `unite()` takes a data frame, the name of the new variable to create, and a set of columns to combine, again specified in `dplyr::select()`. The result is shown in [Figure 9-5](#) and in the following code:

```
table5 %>%
  unite(new, century, year)
#> # A tibble: 6 × 3
#>   country new      rate
#>   *      <chr> <chr>   <chr>
#> 1 Afghanistan 19_99    745/19987071
#> 2 Afghanistan 20_00    2666/20595360
#> 3   Brazil 19_99    37737/172006362
#> 4   Brazil 20_00    80488/174504898
```

```
#> 5      China 19_99 212258/1272915272
#> 6      China 20_00 213766/1280428583
```

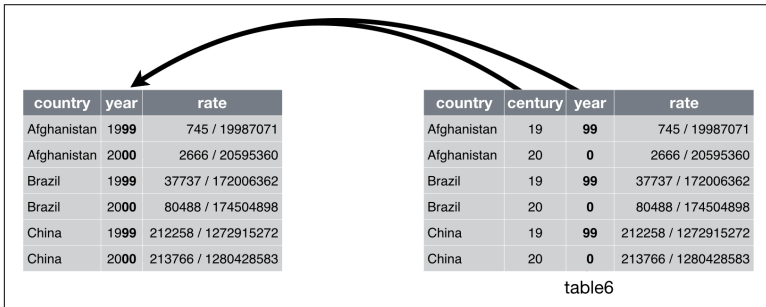


Figure 9-5. Uniting table5 makes it tidy

In this case we also need to use the `sep` argument. The default will place an underscore (`_`) between the values from different columns. Here we don't want any separator so we use `""`:

```
table5 %>%
  unite(new, century, year, sep = "")
#> # A tibble: 6 × 3
#>   country new          rate
#> *   <chr> <chr>      <chr>
#> 1 Afghanistan 1999      745/19987071
#> 2 Afghanistan 2000     2666/20595360
#> 3   Brazil 1999    37737/172006362
#> 4   Brazil 2000    80488/174504898
#> 5     China 1999   212258/1272915272
#> 6     China 2000   213766/1280428583
```

Exercises

1. What do the `extra` and `fill` arguments do in `separate()`? Experiment with the various options for the following two toy datasets:

```
tibble(x = c("a,b,c", "d,e,f,g", "h,i,j")) %>%
  separate(x, c("one", "two", "three"))
```

```
tibble(x = c("a,b,c", "d,e", "f,g,i")) %>%
  separate(x, c("one", "two", "three"))
```

2. Both `unite()` and `separate()` have a `remove` argument. What does it do? Why would you set it to `FALSE`?

3. Compare and contrast `separate()` and `extract()`. Why are there three variations of separation (by position, by separator, and with groups), but only one unite?

Missing Values

Changing the representation of a dataset brings up an important subtlety of missing values. Surprisingly, a value can be missing in one of two possible ways:

- *Explicitly*, i.e., flagged with NA.
- *Implicitly*, i.e., simply not present in the data.

Let's illustrate this idea with a very simple dataset:

```
stocks <- tibble(  
  year = c(2015, 2015, 2015, 2015, 2016, 2016, 2016),  
  qtr = c( 1, 2, 3, 4, 2, 3, 4),  
  return = c(1.88, 0.59, 0.35, NA, 0.92, 0.17, 2.66)  
)
```

There are two missing values in this dataset:

- The return for the fourth quarter of 2015 is explicitly missing, because the cell where its value should be instead contains NA.
- The return for the first quarter of 2016 is implicitly missing, because it simply does not appear in the dataset.

One way to think about the difference is with this Zen-like koan: an explicit missing value is the presence of an absence; an implicit missing value is the absence of a presence.

The way that a dataset is represented can make implicit values explicit. For example, we can make the implicit missing value explicit by putting years in the columns:

```
stocks %>%  
  spread(year, return)  
#> # A tibble: 4 × 3  
#>   qtr `2015` `2016`  
#> * <dbl> <dbl> <dbl>  
#> 1 1 1.88 NA  
#> 2 2 0.59 0.92  
#> 3 3 0.35 0.17  
#> 4 4 NA 2.66
```

Because these explicit missing values may not be important in other representations of the data, you can set `na.rm = TRUE` in `gather()` to turn explicit missing values implicit:

```
stocks %>%
  spread(year, return) %>%
  gather(year, return, `2015`:`2016`, na.rm = TRUE)
#> # A tibble: 6 × 3
#>   qtr year return
#>   * <dbl> <chr> <dbl>
#> 1     1  2015  1.88
#> 2     2  2015  0.59
#> 3     3  2015  0.35
#> 4     2  2016  0.92
#> 5     3  2016  0.17
#> 6     4  2016  2.66
```

Another important tool for making missing values explicit in tidy data is `complete()`:

```
stocks %>%
  complete(year, qtr)
#> # A tibble: 8 × 3
#>   year qtr return
#>   <dbl> <dbl> <dbl>
#> 1  2015     1  1.88
#> 2  2015     2  0.59
#> 3  2015     3  0.35
#> 4  2015     4    NA
#> 5  2016     1    NA
#> 6  2016     2  0.92
#> # ... with 2 more rows
```

`complete()` takes a set of columns, and finds all unique combinations. It then ensures the original dataset contains all those values, filling in explicit NAs where necessary.

There's one other important tool that you should know for working with missing values. Sometimes when a data source has primarily been used for data entry, missing values indicate that the previous value should be carried forward:

```
treatment <- tribble(
  ~ person, ~ treatment, ~response,
  "Derrick Whitmore", 1, 7,
  NA, 2, 10,
  NA, 3, 9,
  "Katherine Burke", 1, 4
)
```


You can fill in these missing values with `fill()`. It takes a set of columns where you want missing values to be replaced by the most recent nonmissing value (sometimes called last observation carried forward):

```
treatment %>%
  fill(person)
#> # A tibble: 4 × 3
#>   person treatment response
#>   <chr>      <dbl>    <dbl>
#> 1 Derrick Whitmore      1      7
#> 2 Derrick Whitmore      2     10
#> 3 Derrick Whitmore      3      9
#> 4 Katherine Burke       1      4
```

Exercises

1. Compare and contrast the `fill` arguments to `spread()` and `complete()`.
2. What does the `direction` argument to `fill()` do?

Case Study

To finish off the chapter, let's pull together everything you've learned to tackle a realistic data tidying problem. The `tidyr::who` dataset contains tuberculosis (TB) cases broken down by year, country, age, gender, and diagnosis method. The data comes from the *2014 World Health Organization Global Tuberculosis Report*, available at <http://www.who.int/tb/country/data/download/en/>.

There's a wealth of epidemiological information in this dataset, but it's challenging to work with the data in the form that it's provided:

```
who
#> # A tibble: 7,240 × 60
#>   country iso2 iso3 year new_sp_m014 new_sp_m1524
#>   <chr> <chr> <chr> <int>    <int>    <int>
#> 1 Afghanistan AF AFG 1980      NA      NA
#> 2 Afghanistan AF AFG 1981      NA      NA
#> 3 Afghanistan AF AFG 1982      NA      NA
#> 4 Afghanistan AF AFG 1983      NA      NA
#> 5 Afghanistan AF AFG 1984      NA      NA
#> 6 Afghanistan AF AFG 1985      NA      NA
#> # ... with 7,234 more rows, and 54 more variables:
#> #   new_sp_m2534 <int>, new_sp_m3544 <int>,
#> #   new_sp_m4554 <int>, new_sp_m5564 <int>,
```

```
#> # new_sp_m65 <int>, new_sp_f014 <int>,
#> # new_sp_f1524 <int>, new_sp_f2534 <int>,
#> # new_sp_f3544 <int>, new_sp_f4554 <int>,
#> # new_sp_f5564 <int>, new_sp_f65 <int>,
#> # new_sn_m014 <int>, new_sn_m1524 <int>,
#> # new_sn_m2534 <int>, new_sn_m3544 <int>,
#> # new_sn_m4554 <int>, new_sn_m5564 <int>,
#> # new_sn_m65 <int>, new_sn_f014 <int>,
#> # new_sn_f1524 <int>, new_sn_f2534 <int>,
#> # new_sn_f3544 <int>, new_sn_f4554 <int>,
#> # new_sn_f5564 <int>, new_sn_f65 <int>,
#> # new_ep_m014 <int>, new_ep_m1524 <int>,
#> # new_ep_m2534 <int>, new_ep_m3544 <int>,
#> # new_ep_m4554 <int>, new_ep_m5564 <int>,
#> # new_ep_m65 <int>, new_ep_f014 <int>,
#> # new_ep_f1524 <int>, new_ep_f2534 <int>,
#> # new_ep_f3544 <int>, new_ep_f4554 <int>,
#> # new_ep_f5564 <int>, new_ep_f65 <int>,
#> # newrel_m014 <int>, newrel_m1524 <int>,
#> # newrel_m2534 <int>, newrel_m3544 <int>,
#> # newrel_m4554 <int>, newrel_m5564 <int>,
#> # newrel_m65 <int>, newrel_f014 <int>,
#> # newrel_f1524 <int>, newrel_f2534 <int>,
#> # newrel_f3544 <int>, newrel_f4554 <int>,
#> # newrel_f5564 <int>, newrel_f65 <int>
```

This is a very typical real-life dataset. It contains redundant columns, odd variable codes, and many missing values. In short, who is messy, and we'll need multiple steps to tidy it. Like **dplyr**, **tidyr** is designed so that each function does one thing well. That means in real-life situations you'll usually need to string together multiple verbs into a pipeline.

The best place to start is almost always to gather together the columns that are not variables. Let's have a look at what we've got:

- It looks like `country`, `iso2`, and `iso3` are three variables that redundantly specify the country.
- `year` is clearly also a variable.
- We don't know what all the other columns are yet, but given the structure in the variable names (e.g., `new_sp_m014`, `new_ep_m014`, `new_ep_f014`) these are likely to be values, not variables.

So we need to gather together all the columns from `new_sp_m014` to `newrel_f65`. We don't know what those values represent yet, so we'll

give them the generic name "key". We know the cells represent the count of cases, so we'll use the variable cases. There are a lot of missing values in the current representation, so for now we'll use `na.rm` just so we can focus on the values that are present:

```
who1 <- who %>%
  gather(
    new_sp_m014:newrel_f65, key = "key",
    value = "cases",
    na.rm = TRUE
  )
who1
#> # A tibble: 76,046 × 6
#>   country iso2 iso3 year      key cases
#>   *      <chr> <chr> <chr> <int>    <chr> <int>
#> 1 Afghanistan AF  AFG 1997 new_sp_m014      0
#> 2 Afghanistan AF  AFG 1998 new_sp_m014     30
#> 3 Afghanistan AF  AFG 1999 new_sp_m014      8
#> 4 Afghanistan AF  AFG 2000 new_sp_m014     52
#> 5 Afghanistan AF  AFG 2001 new_sp_m014    129
#> 6 Afghanistan AF  AFG 2002 new_sp_m014     90
#> # ... with 7.604e+04 more rows
```

We can get some hint of the structure of the values in the new key column by counting them:

```
who1 %>%
  count(key)
#> # A tibble: 56 × 2
#>   key      n
#>   <chr> <int>
#> 1 new_ep_f014 1032
#> 2 new_ep_f1524 1021
#> 3 new_ep_f2534 1021
#> 4 new_ep_f3544 1021
#> 5 new_ep_f4554 1017
#> 6 new_ep_f5564 1017
#> # ... with 50 more rows
```

You might be able to parse this out by yourself with a little thought and some experimentation, but luckily we have the data dictionary handy. It tells us:

1. The first three letters of each column denote whether the column contains new or old cases of TB. In this dataset, each column contains new cases.
2. The next two letters describe the type of TB:
 - rel stands for cases of relapse.

- ep stands for cases of extrapulmonary TB.
 - sn stands for cases of pulmonary TB that could not be diagnosed by a pulmonary smear (smear negative).
 - sp stands for cases of pulmonary TB that could be diagnosed by a pulmonary smear (smear positive).
3. The sixth letter gives the sex of TB patients. The dataset groups cases by males (m) and females (f).
 4. The remaining numbers give the age group. The dataset groups cases into seven age groups:
 - 014 = 0–14 years old
 - 1524 = 15–24 years old
 - 2534 = 25–34 years old
 - 3544 = 35–44 years old
 - 4554 = 45–54 years old
 - 5564 = 55–64 years old
 - 65 = 65 or older

We need to make a minor fix to the format of the column names: unfortunately the names are slightly inconsistent because instead of `new_rel` we have `newrel` (it's hard to spot this here but if you don't fix it we'll get errors in subsequent steps). You'll learn about `str_replace()` in [Chapter 11](#), but the basic idea is pretty simple: replace the characters “newrel” with “new_rel”. This makes all variable names consistent:

```
who2 <- who1 %>%
  mutate(key = stringr::str_replace(key, "newrel", "new_rel"))
who2
#> # A tibble: 76,046 × 6
#>   country iso2 iso3 year      key cases
#>   <chr> <chr> <chr> <int>   <chr> <int>
#> 1 Afghanistan AF AFG 1997 new_sp_m014      0
#> 2 Afghanistan AF AFG 1998 new_sp_m014     30
#> 3 Afghanistan AF AFG 1999 new_sp_m014      8
#> 4 Afghanistan AF AFG 2000 new_sp_m014     52
#> 5 Afghanistan AF AFG 2001 new_sp_m014    129
#> 6 Afghanistan AF AFG 2002 new_sp_m014     90
#> # ... with 7.604e+04 more rows
```

We can separate the values in each code with two passes of `separate()`. The first pass will split the codes at each underscore:

```
who3 <- who2 %>%
  separate(key, c("new", "type", "sexage"), sep = "_")
who3
#> # A tibble: 76,046 × 8
#>   country iso2 iso3 year new type sexage cases
#> *   <chr> <chr> <chr> <int> <chr> <chr> <chr> <int>
#> 1 Afghanistan AF AFG 1997 new sp m014 0
#> 2 Afghanistan AF AFG 1998 new sp m014 30
#> 3 Afghanistan AF AFG 1999 new sp m014 8
#> 4 Afghanistan AF AFG 2000 new sp m014 52
#> 5 Afghanistan AF AFG 2001 new sp m014 129
#> 6 Afghanistan AF AFG 2002 new sp m014 90
#> # ... with 7.604e+04 more rows
```

Then we might as well drop the new column because it's constant in this dataset. While we're dropping columns, let's also drop `iso2` and `iso3` since they're redundant:

```
who3 %>%
  count(new)
#> # A tibble: 1 × 2
#>   new n
#>   <chr> <int>
#> 1 new 76046
who4 <- who3 %>%
  select(-new, -iso2, -iso3)
```

Next we'll separate `sexage` into `sex` and `age` by splitting after the first character:

```
who5 <- who4 %>%
  separate(sexage, c("sex", "age"), sep = 1)
who5
#> # A tibble: 76,046 × 6
#>   country year type sex age cases
#> *   <chr> <int> <chr> <chr> <chr> <int>
#> 1 Afghanistan 1997 sp m 014 0
#> 2 Afghanistan 1998 sp m 014 30
#> 3 Afghanistan 1999 sp m 014 8
#> 4 Afghanistan 2000 sp m 014 52
#> 5 Afghanistan 2001 sp m 014 129
#> 6 Afghanistan 2002 sp m 014 90
#> # ... with 7.604e+04 more rows
```

The who dataset is now tidy!

I've shown you the code a piece at a time, assigning each interim result to a new variable. This typically isn't how you'd work interactively. Instead, you'd gradually build up a complex pipe:

```
who %>%
  gather(code, value, new_sp_m014:newrel_f65, na.rm = TRUE) %>%
  mutate(
    code = stringr::str_replace(code, "newrel", "new_rel")
  ) %>%
  separate(code, c("new", "var", "sexage")) %>%
  select(-new, -iso2, -iso3) %>%
  separate(sexage, c("sex", "age"), sep = 1)
```

Exercises

1. In this case study I set `na.rm = TRUE` just to make it easier to check that we had the correct values. Is this reasonable? Think about how missing values are represented in this dataset. Are there implicit missing values? What's the difference between an NA and zero?
2. What happens if you neglect the `mutate()` step? (`mutate(key = stringr::str_replace(key, "newrel", "new_rel"))`).
3. I claimed that `iso2` and `iso3` were redundant with `country`. Confirm this claim.
4. For each country, year, and sex compute the total number of cases of TB. Make an informative visualization of the data.

Nontidy Data

Before we continue on to other topics, it's worth talking briefly about nontidy data. Earlier in the chapter, I used the pejorative term “messy” to refer to nontidy data. That's an oversimplification: there are lots of useful and well-founded data structures that are not tidy data. There are two main reasons to use other data structures:

- Alternative representations may have substantial performance or space advantages.
- Specialized fields have evolved their own conventions for storing data that may be quite different to the conventions of tidy data.

Either of these reasons means you'll need something other than a tibble (or data frame). If your data does fit naturally into a rectangular structure composed of observations and variables, I think tidy data should be your default choice. But there are good reasons to use other structures; tidy data is not the only way. If you'd like to learn more about nontidy data, I'd highly recommend this [thoughtful blog post](#) by Jeff Leek.

Relational Data with dplyr

Introduction

It's rare that a data analysis involves only a single table of data. Typically you have many tables of data, and you must combine them to answer the questions that you're interested in. Collectively, multiple tables of data are called *relational data* because it is the relations, not just the individual datasets, that are important.

Relations are always defined between a pair of tables. All other relations are built up from this simple idea: the relations of three or more tables are always a property of the relations between each pair. Sometimes both elements of a pair can be the same table! This is needed if, for example, you have a table of people, and each person has a reference to their parents.

To work with relational data you need verbs that work with pairs of tables. There are three families of verbs designed to work with relational data:

- *Mutating joins*, which add new variables to one data frame from matching observations in another.
- *Filtering joins*, which filter observations from one data frame based on whether or not they match an observation in the other table.
- *Set operations*, which treat observations as if they were set elements.

The most common place to find relational data is in a *relational* database management system (or RDBMS), a term that encompasses almost all modern databases. If you’ve used a database before, you’ve almost certainly used SQL. If so, you should find the concepts in this chapter familiar, although their expression in **dplyr** is a little different. Generally, **dplyr** is a little easier to use than SQL because **dplyr** is specialized to do data analysis: it makes common data analysis operations easier, at the expense of making it more difficult to do other things that aren’t commonly needed for data analysis.

Prerequisites

We will explore relational data from **nycflights13** using the two-table verbs from **dplyr**.

```
library(tidyverse)
library(nycflights13)
```

nycflights13

We will use the **nycflights13** package to learn about relational data. **nycflights13** contains four tibbles that are related to the flights table that you used in [Chapter 3](#):

- **airlines** lets you look up the full carrier name from its abbreviated code:

```
airlines
#> # A tibble: 16 × 2
#>   carrier      name
#>   <chr>      <chr>
#> 1     9E Endeavor Air Inc.
#> 2     AA American Airlines Inc.
#> 3     AS  Alaska Airlines Inc.
#> 4     B6   JetBlue Airways
#> 5     DL   Delta Air Lines Inc.
#> 6     EV ExpressJet Airlines Inc.
#> # ... with 10 more rows
```

- **airports** gives information about each airport, identified by the faa airport code:

airports

```
#> # A tibble: 1,396 × 7
#>   faa      name lat lon
#>   <chr>    <chr> <dbl> <dbl>
#> 1 04G      Lansdowne Airport 41.1 -80.6
#> 2 06A      Moton Field Municipal Airport 32.5 -85.7
#> 3 06C      Schaumburg Regional 42.0 -88.1
#> 4 06N      Randall Airport 41.4 -74.4
#> 5 09J      Jekyll Island Airport 31.1 -81.4
#> 6 0A9      Elizabethton Municipal Airport 36.4 -82.2
#> # ... with 1,390 more rows, and 3 more variables:
#> #   alt <int>, tz <dbl>, dst <chr>
```

- planes gives information about each plane, identified by its tailnum:

planes

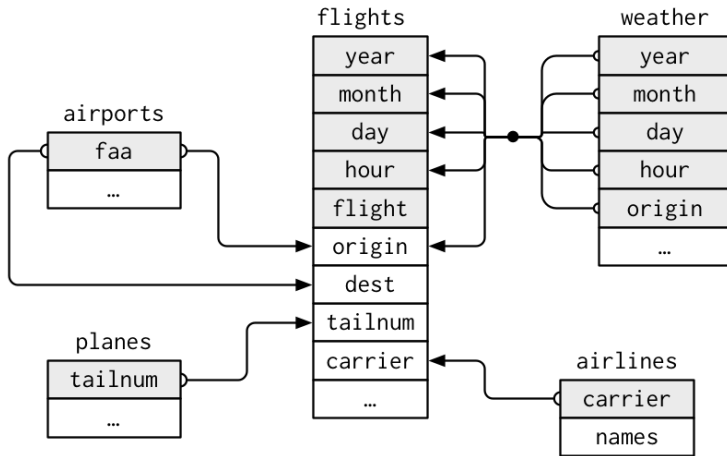
```
#> # A tibble: 3,322 × 9
#>   tailnum year      type
#>   <chr> <int>    <chr>
#> 1 N10156 2004 Fixed wing multi engine
#> 2 N102UW 1998 Fixed wing multi engine
#> 3 N103US 1999 Fixed wing multi engine
#> 4 N104UW 1999 Fixed wing multi engine
#> 5 N10575 2002 Fixed wing multi engine
#> 6 N105UW 1999 Fixed wing multi engine
#> # ... with 3,316 more rows, and 6 more variables:
#> #   manufacturer <chr>, model <chr>, engines <int>,
#> #   seats <int>, speed <int>, engine <chr>
```

- weather gives the weather at each NYC airport for each hour:

weather

```
#> # A tibble: 26,130 × 15
#>   origin year month day hour temp dewp humid
#>   <chr> <dbl> <dbl> <int> <int> <dbl> <dbl> <dbl>
#> 1 EWR 2013 1 1 0 37.0 21.9 54.0
#> 2 EWR 2013 1 1 1 37.0 21.9 54.0
#> 3 EWR 2013 1 1 2 37.9 21.9 52.1
#> 4 EWR 2013 1 1 3 37.9 23.0 54.5
#> 5 EWR 2013 1 1 4 37.9 24.1 57.0
#> 6 EWR 2013 1 1 6 39.0 26.1 59.4
#> # ... with 2.612e+04 more rows, and 7 more variables:
#> #   wind_dir <dbl>, wind_speed <dbl>, wind_gust <dbl>,
#> #   precip <dbl>, pressure <dbl>, visib <dbl>,
#> #   time_hour <dtm>
```

One way to show the relationships between the different tables is with a drawing:



This diagram is a little overwhelming, but it's simple compared to some you'll see in the wild! The key to understanding diagrams like this is to remember each relation always concerns a pair of tables. You don't need to understand the whole thing; you just need to understand the chain of relations between the tables that you are interested in.

For **nycflights13**:

- **flights** connects to **planes** via a single variable, `tailnum`.
- **flights** connects to **airlines** through the `carrier` variable.
- **flights** connects to **airports** in two ways: via the `origin` and `dest` variables.
- **flights** connects to **weather** via `origin` (the location), and `year`, `month`, `day`, and `hour` (the time).

Exercises

1. Imagine you wanted to draw (approximately) the route each plane flies from its origin to its destination. What variables would you need? What tables would you need to combine?
2. I forgot to draw the relationship between **weather** and **airports**. What is the relationship and how should it appear in the diagram?

3. `weather` only contains information for the origin (NYC) airports. If it contained weather records for all airports in the USA, what additional relation would it define with `flights`?
4. We know that some days of the year are “special,” and fewer people than usual fly on them. How might you represent that data as a data frame? What would be the primary keys of that table? How would it connect to the existing tables?

Keys

The variables used to connect each pair of tables are called *keys*. A key is a variable (or set of variables) that uniquely identifies an observation. In simple cases, a single variable is sufficient to identify an observation. For example, each plane is uniquely identified by its `tailnum`. In other cases, multiple variables may be needed. For example, to identify an observation in `weather` you need five variables: `year`, `month`, `day`, `hour`, and `origin`.

There are two types of keys:

- A *primary key* uniquely identifies an observation in its own table. For example, `planes$tailnum` is a primary key because it uniquely identifies each plane in the `planes` table.
- A *foreign key* uniquely identifies an observation in another table. For example, `flights$tailnum` is a foreign key because it appears in the `flights` table where it matches each flight to a unique plane.

A variable can be both a primary key *and* a foreign key. For example, `origin` is part of the `weather` primary key, and is also a foreign key for the `airport` table.

Once you’ve identified the primary keys in your tables, it’s good practice to verify that they do indeed uniquely identify each observation. One way to do that is to `count()` the primary keys and look for entries where `n` is greater than one:

```
planes %>%  
  count(tailnum) %>%  
  filter(n > 1)  
#> # A tibble: 0 × 2  
#> # ... with 2 variables: tailnum <chr>, n <int>
```

```

weather %>%
  count(year, month, day, hour, origin) %>%
  filter(n > 1)
#> Source: local data frame [0 x 6]
#> Groups: year, month, day, hour [0]
#>
#> # ... with 6 variables: year <dbl>, month <dbl>, day <int>,
#> #   hour <int>, origin <chr>, n <int>

```

Sometimes a table doesn't have an explicit primary key: each row is an observation, but no combination of variables reliably identifies it. For example, what's the primary key in the `flights` table? You might think it would be the date plus the flight or tail number, but neither of those are unique:

```

flights %>%
  count(year, month, day, flight) %>%
  filter(n > 1)
#> Source: local data frame [29,768 x 5]
#> Groups: year, month, day [365]
#>
#>   year month   day flight     n
#>   <int> <int> <int> <int> <int>
#> 1  2013     1     1      1     2
#> 2  2013     1     1      3     2
#> 3  2013     1     1      4     2
#> 4  2013     1     1     11     3
#> 5  2013     1     1     15     2
#> 6  2013     1     1     21     2
#> # ... with 2.976e+04 more rows

```

```

flights %>%
  count(year, month, day, tailnum) %>%
  filter(n > 1)
#> Source: local data frame [64,928 x 5]
#> Groups: year, month, day [365]
#>
#>   year month   day tailnum     n
#>   <int> <int> <int>   <chr> <int>
#> 1  2013     1     1 NOEGMQ     2
#> 2  2013     1     1 N11189     2
#> 3  2013     1     1 N11536     2
#> 4  2013     1     1 N11544     3
#> 5  2013     1     1 N11551     2
#> 6  2013     1     1 N12540     2
#> # ... with 6.492e+04 more rows

```

When starting to work with this data, I had naively assumed that each flight number would be only used once per day: that would make it much easier to communicate problems with a specific flight. Unfortunately that is not the case! If a table lacks a primary key, it's sometimes useful to add one with `mutate()` and `row_number()`. That makes it easier to match observations if you've done some filtering and want to check back in with the original data. This is called a *surrogate key*.

A primary key and the corresponding foreign key in another table form a *relation*. Relations are typically one-to-many. For example, each flight has one plane, but each plane has many flights. In other data, you'll occasionally see a 1-to-1 relationship. You can think of this as a special case of 1-to-many. You can model many-to-many relations with a many-to-1 relation plus a 1-to-many relation. For example, in this data there's a many-to-many relationship between airlines and airports: each airline flies to many airports; each airport hosts many airlines.

Exercises

1. Add a surrogate key to `flights`.
2. Identify the keys in the following datasets:
 - a. `Lahman::Batting`
 - b. `babynames::babynames`
 - c. `nasaweather::atmos`
 - d. `fueleconomy::vehicles`
 - e. `ggplot2::diamonds`(You might need to install some packages and read some documentation.)
3. Draw a diagram illustrating the connections between the `Batting`, `Master`, and `Salaries` tables in the **Lahman** package. Draw another diagram that shows the relationship between `Master`, `Managers`, and `AwardsManagers`.

How would you characterize the relationship between the `Batting`, `Pitching`, and `Fielding` tables?

Mutating Joins

The first tool we'll look at for combining a pair of tables is the *mutating join*. A mutating join allows you to combine variables from two tables. It first matches observations by their keys, then copies across variables from one table to the other.

Like `mutate()`, the join functions add variables to the right, so if you have a lot of variables already, the new variables won't get printed out. For these examples, we'll make it easier to see what's going on in the examples by creating a narrower dataset:

```
flights2 <- flights %>%
  select(year:day, hour, origin, dest, tailnum, carrier)
flights2
#> # A tibble: 336,776 × 8
#>   year month   day hour origin dest tailnum carrier
#>   <int> <int> <int> <dbl> <chr> <chr>   <chr>   <chr>
#> 1  2013     1     1     5   EWR  IAH  N14228    UA
#> 2  2013     1     1     5   LGA  IAH  N24211    UA
#> 3  2013     1     1     5   JFK  MIA  N619AA    AA
#> 4  2013     1     1     5   JFK  BQN  N804JB    B6
#> 5  2013     1     1     6   LGA  ATL  N668DN    DL
#> 6  2013     1     1     5   EWR  ORD  N39463    UA
#> # ... with 3.368e+05 more rows
```

(Remember, when you're in RStudio, you can also use `View()` to avoid this problem.)

Imagine you want to add the full airline name to the `flights2` data. You can combine the `airlines` and `flights2` data frames with `left_join()`:

```
flights2 %>%
  select(-origin, -dest) %>%
  left_join(airlines, by = "carrier")
#> # A tibble: 336,776 × 7
#>   year month   day hour tailnum carrier
#>   <int> <int> <int> <dbl>   <chr>   <chr>
#> 1  2013     1     1     5  N14228    UA
#> 2  2013     1     1     5  N24211    UA
#> 3  2013     1     1     5  N619AA    AA
#> 4  2013     1     1     5  N804JB    B6
#> 5  2013     1     1     6  N668DN    DL
#> 6  2013     1     1     5  N39463    UA
#> # ... with 3.368e+05 more rows, and 1 more variable:
#> #   name <chr>
```


The result of joining `airlines` to `flights2` is an additional variable: `name`. This is why I call this type of join a mutating join. In this case, you could have got to the same place using `mutate()` and R's base subsetting:

```
flights2 %>%
  select(-origin, -dest) %>%
  mutate(name = airlines$name[match(carrier, airlines$carrier)])
#> # A tibble: 336,776 × 7
#>   year month   day hour tailnum carrier
#>   <int> <int> <int> <dbl> <chr>   <chr>
#> 1  2013     1     1     5 N14228    UA
#> 2  2013     1     1     5 N24211    UA
#> 3  2013     1     1     5 N619AA    AA
#> 4  2013     1     1     5 N804JB    B6
#> 5  2013     1     1     6 N668DN    DL
#> 6  2013     1     1     5 N39463    UA
#> # ... with 3.368e+05 more rows, and 1 more variable:
#> #   name <chr>
```

But this is hard to generalize when you need to match multiple variables, and takes close reading to figure out the overall intent.

The following sections explain, in detail, how mutating joins work. You'll start by learning a useful visual representation of joins. We'll then use that to explain the four mutating join functions: the inner join, and the three outer joins. When working with real data, keys don't always uniquely identify observations, so next we'll talk about what happens when there isn't a unique match. Finally, you'll learn how to tell **dplyr** which variables are the keys for a given join.

Understanding Joins

To help you learn how joins work, I'm going to use a visual representation:

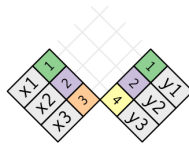
x		y	
1	x1	1	y1
2	x2	2	y2
3	x3	4	y3

```
x <- tribble(
  ~key, ~val_x,
  1, "x1",
  2, "x2",
  3, "x3"
)
```

```
y <- tribble(
  ~key, ~val_y,
  1, "y1",
  2, "y2",
  4, "y3"
)
```

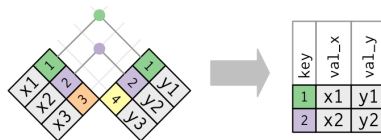
The colored column represents the “key” variable: these are used to match the rows between the tables. The gray column represents the “value” column that is carried along for the ride. In these examples I’ll show a single key variable and single value variable, but the idea generalizes in a straightforward way to multiple keys and multiple values.

A join is a way of connecting each row in *x* to zero, one, or more rows in *y*. The following diagram shows each potential match as an intersection of a pair of lines:



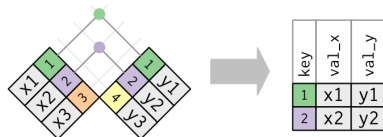
(If you look closely, you might notice that we’ve switched the order of the key and value columns in *x*. This is to emphasize that joins match based on the key; the value is just carried along for the ride.)

In an actual join, matches will be indicated with dots. The number of dots = the number of matches = the number of rows in the output.



Inner Join

The simplest type of join is the *inner join*. An inner join matches pairs of observations whenever their keys are equal:



(To be precise, this is an inner *equijoin* because the keys are matched using the equality operator. Since most joins are equijoins we usually drop that specification.)

The output of an inner join is a new data frame that contains the key, the x values, and the y values. We use `by` to tell **dplyr** which variable is the key:

```
x %>%
  inner_join(y, by = "key")
#> # A tibble: 2 x 3
#>   key val_x val_y
#>   <dbl> <chr> <chr>
#> 1     1     x1    y1
#> 2     2     x2    y2
```

The most important property of an inner join is that unmatched rows are not included in the result. This means that generally inner joins are usually not appropriate for use in analysis because it's too easy to lose observations.

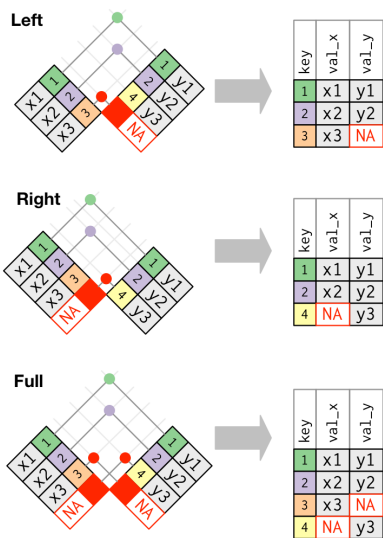
Outer Joins

An inner join keeps observations that appear in both tables. An *outer join* keeps observations that appear in at least one of the tables. There are three types of outer joins:

- A *left join* keeps all observations in x.
- A *right join* keeps all observations in y.
- A *full join* keeps all observations in x and y.

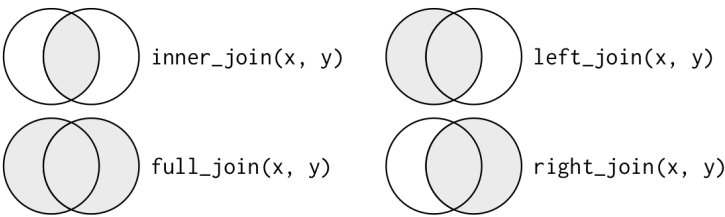
These joins work by adding an additional “virtual” observation to each table. This observation has a key that always matches (if no other key matches), and a value filled with NA.

Graphically, that looks like:



The most commonly used join is the left join: you use this whenever you look up additional data from another table, because it preserves the original observations even when there isn't a match. The left join should be your default join: use it unless you have a strong reason to prefer one of the others.

Another way to depict the different types of joins is with a Venn diagram:

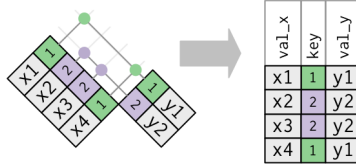


However, this is not a great representation. It might jog your memory about which join preserves the observations in which table, but it suffers from a major limitation: a Venn diagram can't show what happens when keys don't uniquely identify an observation.

Duplicate Keys

So far all the diagrams have assumed that the keys are unique. But that's not always the case. This section explains what happens when the keys are not unique. There are two possibilities:

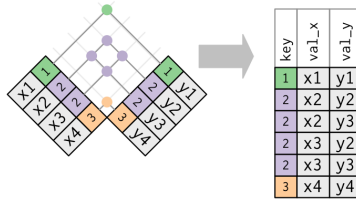
- One table has duplicate keys. This is useful when you want to add in additional information as there is typically a one-to-many relationship:



Note that I've put the key column in a slightly different position in the output. This reflects that the key is a primary key in y and a foreign key in x:

```
x <- tribble(
  ~key, ~val_x,
  1, "x1",
  2, "x2",
  2, "x3",
  1, "x4"
)
y <- tribble(
  ~key, ~val_y,
  1, "y1",
  2, "y2"
)
left_join(x, y, by = "key")
#> # A tibble: 4 × 3
#>   key val_x val_y
#>   <dbl> <chr> <chr>
#> 1     1    x1    y1
#> 2     2    x2    y2
#> 3     2    x3    y2
#> 4     1    x4    y1
```

- Both tables have duplicate keys. This is usually an error because in neither table do the keys uniquely identify an observation. When you join duplicated keys, you get all possible combinations, the Cartesian product:



```
x <- tribble(
  ~key, ~val_x,
  1, "x1",
  2, "x2",
  2, "x3",
  3, "x4"
)
y <- tribble(
  ~key, ~val_y,
  1, "y1",
  2, "y2",
  2, "y3",
  3, "y4"
)
left_join(x, y, by = "key")
#> # A tibble: 6 × 3
#>   key val_x val_y
#>   <dbl> <chr> <chr>
#> 1     1    x1    y1
#> 2     2    x2    y2
#> 3     2    x2    y3
#> 4     2    x3    y2
#> 5     2    x3    y3
#> 6     3    x4    y4
```

Defining the Key Columns

So far, the pairs of tables have always been joined by a single variable, and that variable has the same name in both tables. That constraint was encoded by `by = "key"`. You can use other values for `by` to connect the tables in other ways:

- The default, `by = NULL`, uses all variables that appear in both tables, the so-called *natural* join. For example, the `flights` and `weather` tables match on their common variables: `year`, `month`, `day`, `hour`, and `origin`:

```

flights2 %>%
  left_join(weather)
#> Joining, by = c("year", "month", "day", "hour",
#>   "origin")
#> # A tibble: 336,776 × 18
#>   year month   day hour origin dest tailnum
#>   <dbl> <dbl> <int> <dbl> <chr> <chr>   <chr>
#> 1  2013     1     1     5   EWR   IAH   N14228
#> 2  2013     1     1     5   LGA   IAH   N24211
#> 3  2013     1     1     5   JFK   MIA   N619AA
#> 4  2013     1     1     5   JFK   BQN   N804JB
#> 5  2013     1     1     6   LGA   ATL   N668DN
#> 6  2013     1     1     5   EWR   ORD   N39463
#> # ... with 3.368e+05 more rows, and 11 more variables:
#> #   carrier <chr>, temp <dbl>, dewp <dbl>,
#> #   humid <dbl>, wind_dir <dbl>, wind_speed <dbl>,
#> #   wind_gust <dbl>, precip <dbl>, pressure <dbl>,
#> #   visib <dbl>, time_hour <dtm>

```

- A character vector, `by = "x"`. This is like a natural join, but uses only some of the common variables. For example, `flights` and `planes` have `year` variables, but they mean different things so we only want to join by `tailnum`:

```

flights2 %>%
  left_join(planes, by = "tailnum")
#> # A tibble: 336,776 × 16
#>   year.x month   day hour origin dest tailnum
#>   <int> <int> <int> <dbl> <chr> <chr>   <chr>
#> 1  2013     1     1     5   EWR   IAH   N14228
#> 2  2013     1     1     5   LGA   IAH   N24211
#> 3  2013     1     1     5   JFK   MIA   N619AA
#> 4  2013     1     1     5   JFK   BQN   N804JB
#> 5  2013     1     1     6   LGA   ATL   N668DN
#> 6  2013     1     1     5   EWR   ORD   N39463
#> # ... with 3.368e+05 more rows, and 9 more variables:
#> #   carrier <chr>, year.y <int>, type <chr>,
#> #   manufacturer <chr>, model <chr>, engines <int>,
#> #   seats <int>, speed <int>, engine <chr>

```

Note that the `year` variables (which appear in both input data frames, but are not constrained to be equal) are disambiguated in the output with a suffix.

- A named character vector: `by = c("a" = "b")`. This will match variable `a` in table `x` to variable `b` in table `y`. The variables from `x` will be used in the output.

For example, if we want to draw a map we need to combine the flights data with the airports data, which contains the location (lat and long) of each airport. Each flight has an origin and destination airport, so we need to specify which one we want to join to:

```
flights2 %>%
  left_join(airports, c("dest" = "faa"))
#> # A tibble: 336,776 × 14
#>   year month   day hour origin dest tailnum
#>   <int> <int> <int> <dbl> <chr> <chr>   <chr>
#> 1  2013     1     1     5   EWR   IAH   N14228
#> 2  2013     1     1     5   LGA   IAH   N24211
#> 3  2013     1     1     5   JFK   MIA   N619AA
#> 4  2013     1     1     5   JFK   BQN   N804JB
#> 5  2013     1     1     6   LGA   ATL   N668DN
#> 6  2013     1     1     5   EWR   ORD   N39463
#> # ... with 3.368e+05 more rows, and 7 more variables:
#> #   carrier <chr>, name <chr>, lat <dbl>, lon <dbl>,
#> #   alt <int>, tz <dbl>, dst <chr>

flights2 %>%
  left_join(airports, c("origin" = "faa"))
#> # A tibble: 336,776 × 14
#>   year month   day hour origin dest tailnum
#>   <int> <int> <int> <dbl> <chr> <chr>   <chr>
#> 1  2013     1     1     5   EWR   IAH   N14228
#> 2  2013     1     1     5   LGA   IAH   N24211
#> 3  2013     1     1     5   JFK   MIA   N619AA
#> 4  2013     1     1     5   JFK   BQN   N804JB
#> 5  2013     1     1     6   LGA   ATL   N668DN
#> 6  2013     1     1     5   EWR   ORD   N39463
#> # ... with 3.368e+05 more rows, and 7 more variables:
#> #   carrier <chr>, name <chr>, lat <dbl>, lon <dbl>,
#> #   alt <int>, tz <dbl>, dst <chr>
```

Exercises

1. Compute the average delay by destination, then join on the airports data frame so you can show the spatial distribution of delays. Here's an easy way to draw a map of the United States:

```
airports %>%
  semi_join(flights, c("faa" = "dest")) %>%
  ggplot(aes(lon, lat)) +
  borders("state") +
```



```
geom_point() +  
coord_quickmap()
```

(Don't worry if you don't understand what `semi_join()` does—you'll learn about it next.)

You might want to use the `size` or `color` of the points to display the average delay for each airport.

2. Add the location of the origin *and* destination (i.e., the `lat` and `lon`) to `flights`.
3. Is there a relationship between the age of a plane and its delays?
4. What weather conditions make it more likely to see a delay?
5. What happened on June 13, 2013? Display the spatial pattern of delays, and then use Google to cross-reference with the weather.

Other Implementations

`base::merge()` can perform all four types of mutating join:

dplyr	merge
<code>inner_join(x, y)</code>	<code>merge(x, y)</code>
<code>left_join(x, y)</code>	<code>merge(x, y, all.x = TRUE)</code>
<code>right_join(x, y)</code>	<code>merge(x, y, all.y = TRUE),</code>
<code>full_join(x, y)</code>	<code>merge(x, y, all.x = TRUE, all.y = TRUE)</code>

The advantages of the specific **dplyr** verbs is that they more clearly convey the intent of your code: the difference between the joins is really important but concealed in the arguments of `merge()`. **dplyr**'s joins are considerably faster and don't mess with the order of the rows.

SQL is the inspiration for **dplyr**'s conventions, so the translation is straightforward:

dplyr	SQL
<code>inner_join(x, y, by = "z")</code>	<code>SELECT * FROM x INNER JOIN y USING (z)</code>
<code>left_join(x, y, by = "z")</code>	<code>SELECT * FROM x LEFT OUTER JOIN y USING (z)</code>

dplyr	SQL
<code>right_join(x, y, by = "z")</code>	<code>SELECT * FROM x RIGHT OUTER JOIN y USING (z)</code>
<code>full_join(x, y, by = "z")</code>	<code>SELECT * FROM x FULL OUTER JOIN y USING (z)</code>

Note that "INNER" and "OUTER" are optional, and often omitted.

Joining different variables between the tables, e.g., `inner_join(x, y, by = c("a" = "b"))`, uses a slightly different syntax in SQL: `SELECT * FROM x INNER JOIN y ON x.a = y.b`. As this syntax suggests, SQL supports a wider range of join types than **dplyr** because you can connect the tables using constraints other than equality (sometimes called non-equi joins).

Filtering Joins

Filtering joins match observations in the same way as mutating joins, but affect the observations, not the variables. There are two types:

- `semi_join(x, y)` *keeps* all observations in `x` that have a match in `y`.
- `anti_join(x, y)` *drops* all observations in `x` that have a match in `y`.

Semi-joins are useful for matching filtered summary tables back to the original rows. For example, imagine you've found the top-10 most popular destinations:

```
top_dest <- flights %>%
  count(dest, sort = TRUE) %>%
  head(10)
top_dest
#> # A tibble: 10 × 2
#>   dest      n
#>   <chr> <int>
#> 1 ORD  17283
#> 2 ATL  17215
#> 3 LAX  16174
#> 4 BOS  15508
#> 5 MCO  14082
#> 6 CLT  14064
#> # ... with 4 more rows
```

Now you want to find each flight that went to one of those destinations. You could construct a filter yourself:

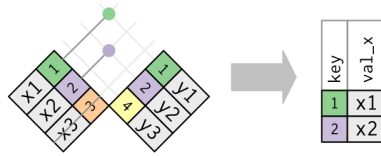
```
flights %>%
  filter(dest %in% top_dest$dest)
#> # A tibble: 141,145 × 19
#>   year month   day dep_time sched_dep_time dep_delay
#>   <int> <int> <int>   <int>         <int>     <dbl>
#> 1  2013     1     1     542             540         2
#> 2  2013     1     1     554             600        -6
#> 3  2013     1     1     554             558        -4
#> 4  2013     1     1     555             600        -5
#> 5  2013     1     1     557             600        -3
#> 6  2013     1     1     558             600        -2
#> # ... with 1.411e+05 more rows, and 12 more variables:
#> #   arr_time <int>, sched_arr_time <int>, arr_delay <dbl>,
#> #   carrier <chr>, flight <int>, tailnum <chr>, origin <chr>,
#> #   dest <chr>, air_time <dbl>, distance <dbl>, hour <dbl>,
#> #   minute <dbl>, time_hour <dtm>
```

But it's difficult to extend that approach to multiple variables. For example, imagine that you'd found the 10 days with the highest average delays. How would you construct the filter statement that used year, month, and day to match it back to flights?

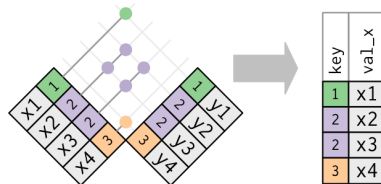
Instead you can use a semi-join, which connects the two tables like a mutating join, but instead of adding new columns, only keeps the rows in *x* that have a match in *y*:

```
flights %>%
  semi_join(top_dest)
#> Joining, by = "dest"
#> # A tibble: 141,145 × 19
#>   year month   day dep_time sched_dep_time dep_delay
#>   <int> <int> <int>   <int>         <int>     <dbl>
#> 1  2013     1     1     554             558        -4
#> 2  2013     1     1     558             600        -2
#> 3  2013     1     1     608             600         8
#> 4  2013     1     1     629             630        -1
#> 5  2013     1     1     656             700        -4
#> 6  2013     1     1     709             700         9
#> # ... with 1.411e+05 more rows, and 13 more variables:
#> #   arr_time <int>, sched_arr_time <int>, arr_delay <dbl>,
#> #   carrier <chr>, flight <int>, tailnum <chr>, origin <chr>,
#> #   dest <chr>, air_time <dbl>, distance <dbl>, hour <dbl>,
#> #   minute <dbl>, time_hour <dtm>
```

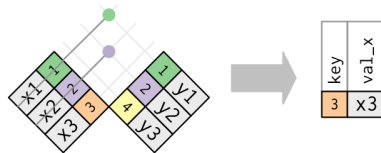
Graphically, a semi-join looks like this:



Only the existence of a match is important; it doesn't matter which observation is matched. This means that filtering joins never duplicate rows like mutating joins do:



The inverse of a semi-join is an anti-join. An anti-join keeps the rows that *don't* have a match:



Anti-joins are useful for diagnosing join mismatches. For example, when connecting flights and planes, you might be interested to know that there are many flights that don't have a match in planes:

```
flights %>%
  anti_join(planes, by = "tailnum") %>%
  count(tailnum, sort = TRUE)
#> # A tibble: 722 × 2
#>   tailnum      n
#>   <chr> <int>
#> 1 <NA>    2512
#> 2 N725MQ     575
#> 3 N722MQ     513
#> 4 N723MQ     507
#> 5 N713MQ     483
```

```
#> 6 N735MQ 396  
#> # ... with 716 more rows
```

Exercises

1. What does it mean for a flight to have a missing `tailnum`? What do the tail numbers that don't have a matching record in `planes` have in common? (Hint: one variable explains ~90% of the problems.)
2. Filter flights to only show flights with planes that have flown at least 100 flights.
3. Combine `fueleconomy::vehicles` and `fueleconomy::common` to find only the records for the most common models.
4. Find the 48 hours (over the course of the whole year) that have the worst delays. Cross-reference it with the `weather` data. Can you see any patterns?
5. What does `anti_join(flights, airports, by = c("dest" = "faa"))` tell you? What does `anti_join(airports, flights, by = c("faa" = "dest"))` tell you?
6. You might expect that there's an implicit relationship between plane and airline, because each plane is flown by a single airline. Confirm or reject this hypothesis using the tools you've learned in the preceding section.

Join Problems

The data you've been working with in this chapter has been cleaned up so that you'll have as few problems as possible. Your own data is unlikely to be so nice, so there are a few things that you should do with your own data to make your joins go smoothly:

1. Start by identifying the variables that form the primary key in each table. You should usually do this based on your understanding of the data, not empirically by looking for a combination of variables that give a unique identifier. If you just look for variables without thinking about what they mean, you might get (un)lucky and find a combination that's unique in your current data but the relationship might not be true in general.

For example, the altitude and longitude uniquely identify each airport, but they are not good identifiers!

```
airports %>% count(alt, lon) %>% filter(n > 1)
#> Source: local data frame [0 x 3]
#> Groups: alt [0]
#>
#> # ... with 3 variables: alt <int>, lon <dbl>, n <int>
```

2. Check that none of the variables in the primary key are missing. If a value is missing then it can't identify an observation!
3. Check that your foreign keys match primary keys in another table. The best way to do this is with an `anti_join()`. It's common for keys not to match because of data entry errors. Fixing these is often a lot of work.

If you do have missing keys, you'll need to be thoughtful about your use of inner versus outer joins, carefully considering whether or not you want to drop rows that don't have a match.

Be aware that simply checking the number of rows before and after the join is not sufficient to ensure that your join has gone smoothly. If you have an inner join with duplicate keys in both tables, you might get unlucky as the number of dropped rows might exactly equal the number of duplicated rows!

Set Operations

The final type of two-table verb are the set operations. Generally, I use these the least frequently, but they are occasionally useful when you want to break a single complex filter into simpler pieces. All these operations work with a complete row, comparing the values of every variable. These expect the `x` and `y` inputs to have the same variables, and treat the observations like sets:

`intersect(x, y)`

Return only observations in both `x` and `y`.

`union(x, y)`

Return unique observations in `x` and `y`.

`setdiff(x, y)`

Return observations in `x`, but not in `y`.

Given this simple data:

```
df1 <- tribble(
  ~x, ~y,
  1, 1,
  2, 1
)
df2 <- tribble(
  ~x, ~y,
  1, 1,
  1, 2
)
```

The four possibilities are:

```
intersect(df1, df2)
#> # A tibble: 1 × 2
#>       x     y
#>   <dbl> <dbl>
#> 1     1     1

# Note that we get 3 rows, not 4
union(df1, df2)
#> # A tibble: 3 × 2
#>       x     y
#>   <dbl> <dbl>
#> 1     1     2
#> 2     2     1
#> 3     1     1

setdiff(df1, df2)
#> # A tibble: 1 × 2
#>       x     y
#>   <dbl> <dbl>
#> 1     2     1

setdiff(df2, df1)
#> # A tibble: 1 × 2
#>       x     y
#>   <dbl> <dbl>
#> 1     1     2
```

Strings with stringr

Introduction

This chapter introduces you to string manipulation in R. You'll learn the basics of how strings work and how to create them by hand, but the focus of this chapter will be on regular expressions, or *regexps* for short. Regular expressions are useful because strings usually contain unstructured or semi-structured data, and regexps are a concise language for describing patterns in strings. When you first look at a regexp, you'll think a cat walked across your keyboard, but as your understanding improves they will soon start to make sense.

Prerequisites

This chapter will focus on the **stringr** package for string manipulation. **stringr** is not part of the core tidyverse because you don't always have textual data, so we need to load it explicitly.

```
library(tidyverse)
library(stringr)
```

String Basics

You can create strings with either single quotes or double quotes. Unlike other languages, there is no difference in behavior. I recommend always using `"`, unless you want to create a string that contains multiple `"`:

```
string1 <- "This is a string"
string2 <- 'To put a "quote" inside a string, use single quotes'
```

If you forget to close a quote, you'll see +, the continuation character:

```
> "This is a string without a closing quote
+
+
+ HELP I'M STUCK
```

If this happens to you, press Esc and try again!

To include a literal single or double quote in a string you can use \ to “escape” it:

```
double_quote <- "\"" # or '"'
single_quote <- "'" # or '"'
```

That means if you want to include a literal backslash, you'll need to double it up: "\\".

Beware that the printed representation of a string is not the same as string itself, because the printed representation shows the escapes. To see the raw contents of the string, use `writeln()`:

```
x <- c("\\", "\\")
x
#> [1] "\" \"\" \"|\"
writeln(x)
#> "
#> |
```

There are a handful of other special characters. The most common are "\n", newline, and "\t", tab, but you can see the complete list by requesting help on `?\"'`, or `?\"'`. You'll also sometimes see strings like "\u00b5", which is a way of writing non-English characters that works on all platforms:

```
x <- "\u00b5"
x
#> [1] "μ"
```

Multiple strings are often stored in a character vector, which you can create with `c()`:

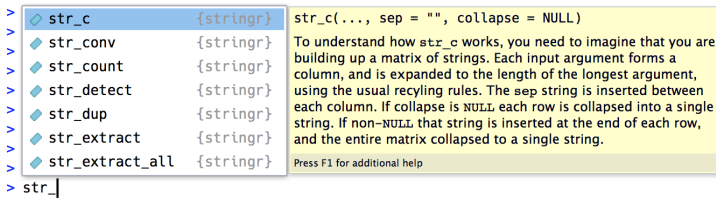
```
c("one", "two", "three")
#> [1] "one" "two" "three"
```

String Length

Base R contains many functions to work with strings but we'll avoid them because they can be inconsistent, which makes them hard to remember. Instead we'll use functions from **stringr**. These have more intuitive names, and all start with `str_`. For example, `str_length()` tells you the number of characters in a string:

```
str_length(c("a", "R for data science", NA))
#> [1]  1 18 NA
```

The common `str_` prefix is particularly useful if you use RStudio, because typing `str_` will trigger autocomplete, allowing you to see all **stringr** functions:



Combining Strings

To combine two or more strings, use `str_c()`:

```
str_c("x", "y")
#> [1] "xy"
str_c("x", "y", "z")
#> [1] "xyz"
```

Use the `sep` argument to control how they're separated:

```
str_c("x", "y", sep = ", ")
#> [1] "x, y"
```

Like most other functions in R, missing values are contagious. If you want them to print as "NA", use `str_replace_na()`:

```
x <- c("abc", NA)
str_c("|-", x, "-|")
#> [1] "|-abc-|" NA
str_c("|-", str_replace_na(x), "-|")
#> [1] "|-abc-|" "|-NA-|"
```

As shown in the preceding code, `str_c()` is vectorized, and it automatically recycles shorter vectors to the same length as the longest:

```
str_c("prefix-", c("a", "b", "c"), "-suffix")
#> [1] "prefix-a-suffix" "prefix-b-suffix" "prefix-c-suffix"
```

Objects of length 0 are silently dropped. This is particularly useful in conjunction with `if`:

```
name <- "Hadley"
time_of_day <- "morning"
birthday <- FALSE

str_c(
  "Good ", time_of_day, " ", name,
  if (birthday) " and HAPPY BIRTHDAY",
  "."
)
#> [1] "Good morning Hadley."
```

To collapse a vector of strings into a single string, use `collapse`:

```
str_c(c("x", "y", "z"), collapse = ", ")
#> [1] "x, y, z"
```

Subsetting Strings

You can extract parts of a string using `str_sub()`. As well as the string, `str_sub()` takes `start` and `end` arguments that give the (inclusive) position of the substring:

```
x <- c("Apple", "Banana", "Pear")
str_sub(x, 1, 3)
#> [1] "App" "Ban" "Pea"

# negative numbers count backwards from end
str_sub(x, -3, -1)
#> [1] "ple" "ana" "ear"
```

Note that `str_sub()` won't fail if the string is too short; it will just return as much as possible:

```
str_sub("a", 1, 5)
#> [1] "a"
```

You can also use the assignment form of `str_sub()` to modify strings:

```
str_sub(x, 1, 1) <- str_to_lower(str_sub(x, 1, 1))
x
#> [1] "apple" "banana" "pear"
```

Locales

Earlier I used `str_to_lower()` to change the text to lowercase. You can also use `str_to_upper()` or `str_to_title()`. However, changing case is more complicated than it might at first appear because different languages have different rules for changing case. You can pick which set of rules to use by specifying a locale:

```
# Turkish has two i's: with and without a dot, and it
# has a different rule for capitalizing them:
str_to_upper(c("i", "ı"))
#> [1] "I" "I"
str_to_upper(c("i", "ı"), locale = "tr")
#> [1] "İ" "I"
```

The locale is specified as an ISO 639 language code, which is a two- or three-letter abbreviation. If you don't already know the code for your language, [Wikipedia](#) has a good list. If you leave the locale blank, it will use the current locale, as provided by your operating system.

Another important operation that's affected by the locale is sorting. The base R `order()` and `sort()` functions sort strings using the current locale. If you want robust behavior across different computers, you may want to use `str_sort()` and `str_order()`, which take an additional `locale` argument:

```
x <- c("apple", "eggplant", "banana")

str_sort(x, locale = "en") # English
#> [1] "apple"      "banana"     "eggplant"

str_sort(x, locale = "haw") # Hawaiian
#> [1] "apple"      "eggplant"   "banana"
```

Exercises

1. In code that doesn't use **stringr**, you'll often see `paste()` and `paste0()`. What's the difference between the two functions? What **stringr** function are they equivalent to? How do the functions differ in their handling of NA?
2. In your own words, describe the difference between the `sep` and `collapse` arguments to `str_c()`.

3. Use `str_length()` and `str_sub()` to extract the middle character from a string. What will you do if the string has an even number of characters?
4. What does `str_wrap()` do? When might you want to use it?
5. What does `str_trim()` do? What's the opposite of `str_trim()`?
6. Write a function that turns (e.g.) a vector `c("a", "b", "c")` into the string `a`, `b`, and `c`. Think carefully about what it should do if given a vector of length 0, 1, or 2.

Matching Patterns with Regular Expressions

Regexps are a very terse language that allow you to describe patterns in strings. They take a little while to get your head around, but once you understand them, you'll find them extremely useful.

To learn regular expressions, we'll use `str_view()` and `str_view_all()`. These functions take a character vector and a regular expression, and show you how they match. We'll start with very simple regular expressions and then gradually get more and more complicated. Once you've mastered pattern matching, you'll learn how to apply those ideas with various **stringr** functions.

Basic Matches

The simplest patterns match exact strings:

```
x <- c("apple", "banana", "pear")
str_view(x, "an")
```

```
apple
banana
pear
```

The next step up in complexity is `.`, which matches any character (except a newline):

```
str_view(x, ".a.")
```

```
apple
banana
peara
```

But if "." matches any character, how do you match the character "."? You need to use an “escape” to tell the regular expression you want to match it exactly, not use its special behavior. Like strings, regexps use the backslash, \, to escape special behavior. So to match an ., you need the regexp \.. Unfortunately this creates a problem. We use strings to represent regular expressions, and \ is also used as an escape symbol in strings. So to create the regular expression \., we need the string "\\.":

```
# To create the regular expression, we need ||
dot <- "\\."

# But the expression itself only contains one:
writeLines(dot)
#> |.

# And this tells R to look for an explicit .
str_view(c("abc", "a.c", "bef"), "a\\.c")

abc
a.c
bef
```

If \ is used as an escape character in regular expressions, how do you match a literal \? Well you need to escape it, creating the regular expression \\. To create that regular expression, you need to use a string, which also needs to escape \. That means to match a literal \ you need to write "\\\\"—you need four backslashes to match one!

```
x <- "a\\b"
writeLines(x)
#> a\b

str_view(x, "\\")

a\b
```

In this book, I'll write regular expressions as \. and strings that represent the regular expression as "\\.".

Exercises

1. Explain why each of these strings don't match a \: "\", "\\\"", "\\ \".

2. How would you match the sequence "'\?
3. What patterns will the regular expression `\.\.\.\.` match? How would you represent it as a string?

Anchors

By default, regular expressions will match any part of a string. It's often useful to *anchor* the regular expression so that it matches from the start or end of the string. You can use:

- `^` to match the start of the string.
- `$` to match the end of the string.

```
x <- c("apple", "banana", "pear")
str_view(x, "^a")
```

```
apple
banana
pear
```

```
str_view(x, "a$")
```

```
apple
banana
pear
```

To remember which is which, try this mnemonic that I learned from [Evan Misshula](#): if you begin with power (^), you end up with money (\$).

To force a regular expression to only match a complete string, anchor it with both `^` and `$`:

```
x <- c("apple pie", "apple", "apple cake")
str_view(x, "apple")
```

```
apple pie
apple
apple cake
```

```
str_view(x, "^apple$")
```



```
apple pie
apple
apple cake
```

You can also match the boundary between words with `\b`. I don't often use this in R, but I will sometimes use it when I'm doing a search in RStudio when I want to find the name of a function that's a component of other functions. For example, I'll search for `\bsum\b` to avoid matching `summarize`, `summary`, `rowsum`, and so on.

Exercises

1. How would you match the literal string `"$^$"`?
2. Given the corpus of common words in `stringr::words`, create regular expressions that find all words that:
 - a. Start with "y".
 - b. End with "x".
 - c. Are exactly three letters long. (Don't cheat by using `str_length(!)`)
 - d. Have seven letters or more.

Since this list is long, you might want to use the `match` argument to `str_view()` to show only the matching or non-matching words.

Character Classes and Alternatives

There are a number of special patterns that match more than one character. You've already seen `.`, which matches any character apart from a newline. There are four other useful tools:

- `\d` matches any digit.
- `\s` matches any whitespace (e.g., space, tab, newline).
- `[abc]` matches a, b, or c.
- `[^abc]` matches anything except a, b, or c.

Remember, to create a regular expression containing `\d` or `\s`, you'll need to escape the `\` for the string, so you'll type `"\\d"` or `"\\s"`.

You can use *alternation* to pick between one or more alternative patterns. For example, `abc|d..f` will match either "abc", or "deaf". Note that the precedence for `|` is low, so that `abc|xyz` matches `abc` or `xyz` not `abcyz` or `abxyz`. Like with mathematical expressions, if precedence ever gets confusing, use parentheses to make it clear what you want:

```
str_view(c("grey", "gray"), "gr(e|a)y")
```

grey

gray

Exercises

1. Create regular expressions to find all words that:
 - a. Start with a vowel.
 - b. Only contain consonants. (Hint: think about matching “not”-vowels.)
 - c. End with `ed`, but not with `eed`.
 - d. End with `ing` or `ize`.
2. Empirically verify the rule “i before e except after c.”
3. Is “q” always followed by a “u”?
4. Write a regular expression that matches a word if it’s probably written in British English, not American English.
5. Create a regular expression that will match telephone numbers as commonly written in your country.

Repetition

The next step up in power involves controlling how many times a pattern matches:

- `?:` 0 or 1
- `+`: 1 or more
- `*`: 0 or more

```
x <- "1888 is the longest year in Roman numerals: MDCCCLXXXVIII"  
str_view(x, "CC?")
```

1888 is the longest year in Roman numerals: MDCCCLXXXVIII

```
str_view(x, "CC+")
```

1888 is the longest year in Roman numerals: MDCCCLXXXVIII

```
str_view(x, 'C[LX]+')
```

1888 is the longest year in Roman numerals: MDCCCLXXXVIII

Note that the precedence of these operators is high, so you can write `colou?r` to match either American or British spellings. That means most uses will need parentheses, like `bana(na)+`.

You can also specify the number of matches precisely:

- `{n}`: exactly *n*
- `{n,}`: *n* or more
- `{,m}`: at most *m*
- `{n,m}`: between *n* and *m*

```
str_view(x, "C{2}")
```

1888 is the longest year in Roman numerals: MDCCCLXXXVIII

```
str_view(x, "C{2,}")
```

1888 is the longest year in Roman numerals: MDCCCLXXXVIII

```
str_view(x, "C{2,3}")
```

1888 is the longest year in Roman numerals: MDCCCLXXXVIII

By default these matches are “greedy”: they will match the longest string possible. You can make them “lazy,” matching the shortest string possible, by putting a `?` after them. This is an advanced feature of regular expressions, but it’s useful to know that it exists:

```
str_view(x, 'C{2,3}?')
```

1888 is the longest year in Roman numerals: MDCCCLXXXVIII

```
str_view(x, 'C[LX]?')
```

1888 is the longest year in Roman numerals: MDCCCLXXXVIII

Exercises

1. Describe the equivalents of `?`, `+`, and `*` in `{m,n}` form.
2. Describe in words what these regular expressions match (read carefully to see if I'm using a regular expression or a string that defines a regular expression):
 - a. `^.*$`
 - b. `"\\{.+\\}"`
 - c. `\\d{4}-\\d{2}-\\d{2}`
 - d. `"\\\\\\\\{4}"`
3. Create regular expressions to find all words that:
 - a. Start with three consonants.
 - b. Have three or more vowels in a row.
 - c. Have two or more vowel-consonant pairs in a row.
4. Solve the beginner regexp crosswords at <https://regexcrossword.com/challenges/beginner>.

Grouping and Backreferences

Earlier, you learned about parentheses as a way to disambiguate complex expressions. They also define “groups” that you can refer to with *backreferences*, like `\\1`, `\\2`, etc. For example, the following regular expression finds all fruits that have a repeated pair of letters:

```
str_view(fruit, "(..)\\1", match = TRUE)
```

```
banana  
coconut  
cucumber  
jujube  
papaya  
salal berry
```

(Shortly, you'll also see how they're useful in conjunction with `str_match()`.)

Exercises

1. Describe, in words, what these expressions will match:
 - a. `(.)\1\1`
 - b. `"(.)\2\1"`
 - c. `(.)\1`
 - d. `"(.)\1.\1"`
 - e. `"(.)\3\2\1"`
2. Construct regular expressions to match words that:
 - a. Start and end with the same character.
 - b. Contain a repeated pair of letters (e.g., “church” contains “ch” repeated twice).
 - c. Contain one letter repeated in at least three places (e.g., “eleven” contains three “e”s).

Tools

Now that you’ve learned the basics of regular expressions, it’s time to learn how to apply them to real problems. In this section you’ll learn a wide array of **stringr** functions that let you:

- Determine which strings match a pattern.
- Find the positions of matches.
- Extract the content of matches.
- Replace matches with new values.
- Split a string based on a match.

A word of caution before we continue: because regular expressions are so powerful, it’s easy to try and solve every problem with a single regular expression. In the words of Jamie Zawinski:

Some people, when confronted with a problem, think “I know, I’ll use regular expressions.” Now they have two problems.

As a cautionary tale, check out this regular expression that checks if an email address is valid:

Detect Matches

To determine if a character vector matches a pattern, use `str_detect()`. It returns a logical vector the same length as the input:

```
x <- c("apple", "banana", "pear")
str_detect(x, "e")
#> [1] TRUE FALSE TRUE
```

Remember that when you use a logical vector in a numeric context, `FALSE` becomes 0 and `TRUE` becomes 1. That makes `sum()` and `mean()` useful if you want to answer questions about matches across a larger vector:

```
# How many common words start with t?
sum(str_detect(words, "^t"))
#> [1] 65
# What proportion of common words end with a vowel?
mean(str_detect(words, "[aeiou]$"))
#> [1] 0.277
```

When you have complex logical conditions (e.g., match a or b but not c unless d) it's often easier to combine multiple `str_detect()` calls with logical operators, rather than trying to create a single regular expression. For example, here are two ways to find all words that don't contain any vowels:

```
# Find all words containing at least one vowel, and negate
no_vowels_1 <- !str_detect(words, "[aeiou]")
# Find all words consisting only of consonants (non-vowels)
no_vowels_2 <- str_detect(words, "^[^aeiou]+$")
identical(no_vowels_1, no_vowels_2)
#> [1] TRUE
```

The results are identical, but I think the first approach is significantly easier to understand. If your regular expression gets overly complicated, try breaking it up into smaller pieces, giving each piece a name, and then combining the pieces with logical operations.

A common use of `str_detect()` is to select the elements that match a pattern. You can do this with logical subsetting, or the convenient `str_subset()` wrapper:

```
words[str_detect(words, "x$")]
#> [1] "box" "sex" "six" "tax"
str_subset(words, "x$")
#> [1] "box" "sex" "six" "tax"
```

Typically, however, your strings will be one column of a data frame, and you'll want to use `filter` instead:

```
df <- tibble(
  word = words,
  i = seq_along(word)
)
df %>%
  filter(str_detect(words, "x$"))
#> # A tibble: 4 x 2
#>   word      i
#>   <chr> <int>
#> 1 box    108
#> 2 sex    747
#> 3 six    772
#> 4 tax    841
```

A variation on `str_detect()` is `str_count()`: rather than a simple yes or no, it tells you how many matches there are in a string:

```
x <- c("apple", "banana", "pear")
str_count(x, "a")
#> [1] 1 3 1

# On average, how many vowels per word?
mean(str_count(words, "[aeiou]"))
#> [1] 1.99
```

It's natural to use `str_count()` with `mutate()`:

```
df %>%
  mutate(
    vowels = str_count(word, "[aeiou]"),
    consonants = str_count(word, "[^aeiou]")
  )
#> # A tibble: 980 x 4
#>   word      i vowels consonants
#>   <chr> <int> <int>      <int>
#> 1 a      1     1         0
#> 2 able    2     2         2
#> 3 about   3     3         2
#> 4 absolute 4     4         4
#> 5 accept  5     2         4
#> 6 account 6     3         4
#> # ... with 974 more rows
```

abababa

Note that matches never overlap. For example, in "abababa", how many times will the pattern "aba" match? Regular expressions say two, not three:

```
str_count("abababa", "aba")
#> [1] 2
str_view_all("abababa", "aba")
abababa
```

Note the use of `str_view_all()`. As you'll shortly learn, many **stringr** functions come in pairs: one function works with a single match, and the other works with all matches. The second function will have the suffix `_all`.

Exercises

1. For each of the following challenges, try solving it by using both a single regular expression, and a combination of multiple `str_detect()` calls:
 - a. Find all words that start or end with x.
 - b. Find all words that start with a vowel and end with a consonant.
 - c. Are there any words that contain at least one of each different vowel?
 - d. What word has the highest number of vowels? What word has the highest proportion of vowels? (Hint: what is the denominator?)

Extract Matches

To extract the actual text of a match, use `str_extract()`. To show that off, we're going to need a more complicated example. I'm going to use the **Harvard sentences**, which were designed to test VOIP systems, but are also useful for practicing regexes. These are provided in `stringr::sentences`:

```
length(sentences)
#> [1] 720
head(sentences)
#> [1] "The birch canoe slid on the smooth planks."
#> [2] "Glue the sheet to the dark blue background."
```

```
#> [3] "It's easy to tell the depth of a well."
#> [4] "These days a chicken leg is a rare dish."
#> [5] "Rice is often served in round bowls."
#> [6] "The juice of lemons makes fine punch."
```

Imagine we want to find all sentences that contain a color. We first create a vector of color names, and then turn it into a single regular expression:

```
colors <- c(
  "red", "orange", "yellow", "green", "blue", "purple"
)
color_match <- str_c(colors, collapse = "|")
color_match
#> [1] "red/orange/yellow/green/blue/purple"
```

Now we can select the sentences that contain a color, and then extract the color to figure out which one it is:

```
has_color <- str_subset(sentences, color_match)
matches <- str_extract(has_color, color_match)
head(matches)
#> [1] "blue" "blue" "red" "red" "red" "blue"
```

Note that `str_extract()` only extracts the first match. We can see that most easily by first selecting all the sentences that have more than one match:

```
more <- sentences[str_count(sentences, color_match) > 1]
str_view_all(more, color_match)
```

```
It is hard to erase blue or red ink.
The green light in the brown box flickered.
The sky in the west is tinged with orange red.
```

```
str_extract_all(more, color_match)
#> [1] "blue" "green" "orange"
```

```
It is hard to erase blue or red ink.
The green light in the brown box flickered.
The sky in the west is tinged with orange red.
```

This is a common pattern for **stringr** functions, because working with a single match allows you to use much simpler data structures. To get all matches, use `str_extract_all()`. It returns a list:

```
str_extract_all(more, color_match)
#> [[1]]
#> [1] "blue" "red"
```

```
#>
#> [[2]]
#> [1] "green" "red"
#>
#> [[3]]
#> [1] "orange" "red"
```

You'll learn more about lists in “Recursive Vectors (Lists)” on page 302 and Chapter 17.

If you use `simplify = TRUE`, `str_extract_all()` will return a matrix with short matches expanded to the same length as the longest:

```
str_extract_all(more, color_match, simplify = TRUE)
#>      [,1]      [,2]
#> [1,] "blue"    "red"
#> [2,] "green"   "red"
#> [3,] "orange"  "red"

x <- c("a", "a b", "a b c")
str_extract_all(x, "[a-z]", simplify = TRUE)
#>      [,1] [,2] [,3]
#> [1,] "a"  ""   ""
#> [2,] "a"  "b"  ""
#> [3,] "a"  "b"  "c"
```

Exercises

1. In the previous example, you might have noticed that the regular expression matched “flickered,” which is not a color. Modify the regex to fix the problem.
2. From the Harvard sentences data, extract:
 - a. The first word from each sentence.
 - b. All words ending in `ing`.
 - c. All plurals.

Grouped Matches

Earlier in this chapter we talked about the use of parentheses for clarifying precedence and for backreferences when matching. You can also use parentheses to extract parts of a complex match. For example, imagine we want to extract nouns from the sentences. As a heuristic, we'll look for any word that comes after “a” or “the”. Defining a “word” in a regular expression is a little tricky, so here I use a

simple approximation—a sequence of at least one character that isn't a space:

```
noun <- "(a|the) ([^ ]+)"

has_noun <- sentences %>%
  str_subset(noun) %>%
  head(10)
has_noun %>%
  str_extract(noun)
#> [1] "the smooth" "the sheet" "the depth" "a chicken"
#> [5] "the parked" "the sun" "the huge" "the ball"
#> [9] "the woman" "a helps"
```

`str_extract()` gives us the complete match; `str_match()` gives each individual component. Instead of a character vector, it returns a matrix, with one column for the complete match followed by one column for each group:

```
has_noun %>%
  str_match(noun)
#>      [,1]      [,2] [,3]
#> [1,] "the smooth" "the" "smooth"
#> [2,] "the sheet" "the" "sheet"
#> [3,] "the depth" "the" "depth"
#> [4,] "a chicken" "a" "chicken"
#> [5,] "the parked" "the" "parked"
#> [6,] "the sun" "the" "sun"
#> [7,] "the huge" "the" "huge"
#> [8,] "the ball" "the" "ball"
#> [9,] "the woman" "the" "woman"
#> [10,] "a helps" "a" "helps"
```

(Unsurprisingly, our heuristic for detecting nouns is poor, and also picks up adjectives like *smooth* and *parked*.)

If your data is in a tibble, it's often easier to use `tidyr::extract()`. It works like `str_match()` but requires you to name the matches, which are then placed in new columns:

```
tibble(sentence = sentences) %>%
  tidyr::extract(
    sentence, c("article", "noun"), "(a|the) ([^ ]+)",
    remove = FALSE
  )
#> # A tibble: 720 × 3
#>   sentence article noun
#>   *         <chr>  <chr>
#> 1 The birch canoe slid on the smooth planks. the smooth
#> 2 Glue the sheet to the dark blue background. the sheet
#> 3 It's easy to tell the depth of a well. the depth
```